PHASE 6 · DEVSECOPS LAB BOOK

3-Node HA Kubernetes Control-Plane

From single point of failure to a quorum-based, production-ready control plane.

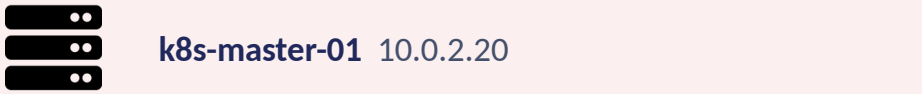
3	2	< 5s	0
Control-Plane Nodes	Worker Nodes	Automatic Failover (RTO)	Data Loss (RPO)

```
Load Balancer VIP: 10.0.2.25:6443
```

ARCHITECTURE CHANGE

Single Control-Plane → 3-Node HA

BEFORE



k8s-master-01 10.0.2.20

Single point of failure — API, scheduler, and etcd all on one node.



k8s-worker-01 10.0.2.21



k8s-worker-02 10.0.2.22

Risk: any master outage takes down the entire cluster API and scheduling.

AFTER



k8s-cp-01 10.0.2.20



k8s-cp-02 10.0.2.21



k8s-cp-03 10.0.2.22

Control-Plane HA Cluster (quorum = 2 of 3)



k8s-worker-01 10.0.2.23



k8s-worker-02 10.0.2.24

Load Balancer VIP `10.0.2.25:6443`

INFRASTRUCTURE

Updated VM Inventory

VM	IP	Role	vCPU	RAM	Disk
k8s-cp-01	10.0.2.20	Control Plane 1 (etcd, API, scheduler)	4	6 GB	60 GB
k8s-cp-02	10.0.2.21	Control Plane 2 (etcd, API, scheduler)	4	6 GB	60 GB
k8s-cp-03	10.0.2.22	Control Plane 3 (etcd, API, scheduler)	4	6 GB	60 GB
k8s-worker-01	10.0.2.23	Kubernetes Worker Node 1	4	8 GB	80 GB
k8s-worker-02	10.0.2.24	Kubernetes Worker Node 2	4	8 GB	80 GB

VCPU

+4

→ 56 total

RAM

+12 GB

→ 94 GB total

DISK

+100 GB

→ 890 GB total

HA ARCHITECTURE

Quorum-Based Availability



etcd Cluster (3 nodes)

- Quorum = 2 of 3
- Can lose 1 node and still function
- Leader election automatic
- Data replicated across all 3



API Server (3 instances)

- Any of 3 can serve requests
- Load balanced behind HAProxy VIP
- Health checks every 1 second
- Automatic failover on timeout



Scheduler (3 instances)

- Only the leader schedules
- Automatic leader election
- Transparent to users

RESILIENCE

Failure Scenarios

Scenario	Result
1 control-plane node down	Cluster continues — 2 replicas remain
1 worker node down	Pods rescheduled to other workers
Network partition	Majority partition survives
Load balancer down	Direct CP IPs still work

RTO

< 5 seconds

Automatic failover

RPO

Zero

etcd replicates synchronously

Prepare Infrastructure & Bootstrap First CP

1 Prepare Infrastructure

```
# Install load balancer on HAProxy node (10.0.2.25)
sudo apt-get install -y haproxy

# VIP 10.0.2.25:6443 → {cp-01, cp-02, cp-03}
# Health checks on TCP:6443 every 1 second
```

HAProxy fronts all 3 control-plane nodes on the same VIP, so kubeadm and kubectl always target a stable endpoint — even mid-failover.

2 Initialize First CP Node

```
# k8s-cp-01 initializes the cluster
kubeadm init --config=kubeadm-config.yaml \
  --control-plane-endpoint="10.0.2.25:6443" \
  --upload-certs

# Saves root CA, etcd certs, CP certs
# Outputs join token (valid 24h)
```

--upload-certs shares control-plane certificates automatically, so joining nodes don't need manual cert copying.

Join Additional Control-Planes & Workers

3 Join CP-02 & CP-03

```
# Each additional CP node joins with:
# --control-plane flag (not just worker)
# --certificate-key (mutual TLS auth)

kubeadm join 10.0.2.25:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --control-plane \
  --certificate-key <key>
```

4 Join Worker Nodes

```
# Workers join WITHOUT --control-plane flag

kubeadm join 10.0.2.25:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```



The only difference between a control-plane join and a worker join is the --control-plane and --certificate-key flags — the token and discovery hash are shared.

Install CNI & Verify

5 Install CNI & Verification

```
kubectl apply -f \
  https://docs.projectcalico.org/manifests/calico.yaml

# All 5 nodes should show: STATUS = Ready
kubectl get nodes
```

QUICK VERIFICATION CHECKS

- `kubectl get pods -A` → no CrashLoopBackOff / Pending
- `kubectl cluster-info` → shows all 3 CP nodes
- `curl -k https://10.0.2.25:6443/api/v1/nodes` → returns JSON

EXPECTED NODE STATUS

k8s-cp-01	Ready
k8s-cp-02	Ready
k8s-cp-03	Ready
k8s-worker-01	Ready
k8s-worker-02	Ready

HA-SPECIFIC SECURITY

Stacked etcd Topology

EACH CP NODE RUNS

kube-apiserver	port 6443
kube-scheduler	port 10259
kube-controller-manager	port 10257
etcd	port 2379

COMMUNICATION PATHS



CP → etcd

localhost (secure)



CP → CP

mTLS (mutual certificate auth)



Workers → API

mTLS (kubelet certificate)

DATA SAFETY

etcd Replication Flow



Quorum Rule

Only written to disk when quorum (2 of 3) acknowledges.

= Zero data loss on failure

Because acknowledgment requires a majority (not all) of nodes, the cluster keeps accepting writes even while one CP node is down or unreachable — at full consistency.

Critical: etcd Backup Procedure

With multiple CP nodes, etcd backup is CRITICAL — it is the single source of truth for the entire cluster state.

```
# Backup etcd from any CP node
sudo su -
export ETCDCTL_API=3
etcdctl --endpoints=127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /backup/etcd-backup-$(date +%Y%m%d-%H%M%S).db

# Keep backups off-cluster (S3, NFS, etc.)
```

Automate this on a schedule (cron / CronJob) and verify snapshot integrity periodically with a restore drill.

GROWTH

Scaling Worker Nodes

With 3 HA control-plane nodes already in place, adding capacity is a one-command operation per node — no control-plane changes required.

```
# For each new worker node (k8s-worker-03, -04, etc.)
# IP: 10.0.2.25+ (increment as needed, max .254)

sudo kubeadm join 10.0.2.25:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

Automatic Join

New workers join the cluster immediately, no manual registration.

Instant Scheduling

Pods can be scheduled on new workers right away.

Horizontal scaling is decoupled from control-plane HA

Because the control plane already tolerates node loss, growing worker capacity never puts cluster availability at risk.

VALIDATION

Post-Setup Verification Checklist

1

All nodes Ready

`kubectl get nodes → 5 nodes with STATUS=Ready`

2

All pods Running

`kubectl get pods -A → no CrashLoopBackOff or Pending`

3

etcd cluster healthy

`etcdctl endpoint health → all 3 endpoints healthy`

4

API server responds

`kubectl cluster-info → shows all 3 CP nodes`

5

Load balancer works

`curl -k https://10.0.2.25:6443/api/v1/nodes → returns JSON`

6

CP failover tested

`Stop kubelet on 1 CP, wait 30s → kubectl still works`

SUMMARY

Single CP vs. 3-Node HA

Aspect	Single CP	3-Node HA
Availability	No HA	**99.95%+ (quorum-based)**
Fault Tolerance	0 nodes	**1 node (can lose 1 of 3)**
Data Safety	Single disk	**Replicated across 3**
RTO	Unknown	**< 5 seconds**
RPO	Data loss possible	**Zero (sync replication)**
Production-Ready	No	**Yes**

UPDATED DOCUMENTATION

Reference Set for 3-Node HA

- 00-MASTER-PLAN.md — Updated VM inventory
- 23-KUBERNETES-CLUSTER.md — Updated installation (HA)
- 24-HELM-PACKAGE-MANAGEMENT.md — Uses updated k8s cluster
- 25-26-ISTIO-GITOPS.md — References 3 CP nodes